

A. Grading Students

Problem Statement:

In this problem, we have to process a list of student grades and apply a rounding rule defined by the university. If a student's grade is close enough to the next multiple of 5 and the grade is not a failing one, it should be rounded up. Otherwise, the original grade remains unchanged. The final grades must be printed after applying these rules

Theory/Approach:

To solve this problem, first we will read all student grades and process them one by one. If a grade is less than 38, it is considered a failing grade and no rounding is applied. And if a grade is equal to 38 or higher, the next multiple of 5 is calculated using integer division. The difference between the grade and this next multiple is checked. If the difference is less than 3, the grade is rounded up, otherwise it is left unchanged

Code:

Lab 2 > Contest > C++ A.cpp > ...

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6
7      int n;
8      cin >> n;
9
10     int grades[n];
11     for(int i=0;i<n;i++){
12         cin >> grades[i];
13     }
14     for(int i=0;i<n;i++){
15
16         int grade = grades[i];
17
18         if (grade<38){
19             cout << grade << endl;
20         }
21         else {
22             int after=((grade/5)+1)*5;
23             int difference=after-grade;
24
25             if (difference<3){
26                 cout << after << endl;
27             } else {
28                 cout << grade << endl;
29             }
30         }
31     }
32     return 0;
33 }
```

Output:

4
73
67
38
33
75
67
40
33

Verdict:

66360101	fardinnuman	A	Accepted	113	583	C++	12 days ago
----------	-------------	---	----------	-----	-----	-----	-------------

Discussion:

Time Complexity: $O(n)$, since each grade is processed once

Space Complexity: $O(n)$, as an array is used to store the grades.

B. Jumping on the Clouds

Problem Statement:

In this problem, a character jumps through a circular sequence of clouds. Each jump consumes energy and landing on certain clouds reduces additional energy. Starting from the first cloud with full energy, the character continues jumping with a fixed jump length until it lands back on the starting cloud. The task is to calculate the remaining energy after completing the full cycle

Theory/Approach:

The solution simulates the movement of the character step by step. The character starts at index 0 with an initial energy of 100. On every jump, the position is updated using modulo arithmetic to maintain the circular path. Each jump reduces energy by 1 unit, and if the character lands on a thundercloud, an additional 2 units are subtracted. The process continues until the character returns to the starting cloud

Code:

Lab 2 > Contest > C++ B.cpp > ...

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      int n, k;
8      cin >> n >> k;
9
10     int c[n];
11     for (int i = 0; i < n; i++)
12     {
13         cin >> c[i];
14     }
15
16     int energy = 100;
17     int position = 0;
18
19     do
20     {
21         position = (position + k) % n;
22         energy -= 1;
23
24         if (c[position] == 1)
25         {
26             energy -= 2;
27         }
28     } while (position != 0);
29
30     cout << energy << endl;
31
32     return 0;
33 }
34 }
```

Output:

```
8 2
0 0 1 0 0 1 1 0
92
```

Verdict:

66626242	fardinnuman	B	Accepted	81	440	C++	22 hr ago
----------	-------------	---	----------	----	-----	-----	-----------

Discussion:

- **Time Complexity:** $O(n / k)$, which in the worst case becomes $O(n)$ when all clouds are visited
- **Space Complexity:** $O(n)$ for storing the cloud configuration
- **Insights:** Modulo operation is crucial to make the cloud path circular and simulating the jumps step by step ensures accurate energy tracking

C. Separate the Numbers

Problem Statement:

A numeric string is considered beautiful if it can be split into a sequence of two or more positive integers such that each number is exactly 1 greater than the previous one, none have leading zeros and the sequence appears in order in the string. The task is to check whether a given string is beautiful and if so, print YES x where x is the first number and otherwise, print NO

Theory/Approach:

This solution tries all possible lengths for the first number, then iteratively checks if the remaining string can be formed by consecutive numbers incremented by 1. If a mismatch occurs or a leading zero is detected, it tries the next first number length. If the end of the string is reached successfully, the string is beautiful

Code:

```
Lab 2 > Contest > C++ C.cpp > ...
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  void separateNumbers(string s)
6  {
7      int n = s.size();
8      for (int len = 1; len <= n / 2; len++) {
9          string firstStr = s.substr(0, len);
10         if (firstStr[0] == '0')
11             continue;
12         long long first = stoll(firstStr), num = first;
13         int idx = len;
14         while (idx < n)
15         {
16             num++;
17             string next = to_string(num);
18             if (s.substr(idx, next.size()) != next)
19                 break;
20             idx += next.size();
21         }
22         if (idx == n)
23         {
24             cout << "YES " << first << endl;
25             return;
26         }
27     }
28     cout << "NO" << endl;
29 }
30
31 int main()
32 {
33     int q;
34     cin >> q;
35     while (q--) {
36         string s;
37         cin >> s;
38         separateNumbers(s);
39     }
40     return 0;
41 }
```

Output:

```
4
99910001001
YES 999
7891011
YES 7
9899100
YES 98
999100010001
NO
```

Verdict:

66626323	fardinnuman	C	Accepted	204	829	C++	22 hr ago
----------	-------------	---	----------	-----	-----	-----	-----------

Discussion:

- **Time Complexity:** $O(n^2)$, checking up to $n/2$ first number lengths
- **Space Complexity:** $O(1)$ extra space
- **Insights:** Leading zeros break sequences; using substrings and stoll ensures correct handling of multi-digit numbers

D. Larry's Array

Problem Statement:

In this problem, we are given a permutation of natural numbers from 1 to n , the task is to determine if it can be sorted into ascending order using only a specific operation: rotating any three consecutive elements. For each test case, print YES if the array can be fully sorted using this operation any number of times; otherwise, print NO

Theory/Approach:

The idea is to count the number of inversions in the array. An inversion occurs when a larger number appears before a smaller one. The given rotation operation does not change whether the number of inversions is even or odd. Since a sorted array has zero (even) inversions, the array can be sorted only if its inversion count is even. Therefore, we count inversions and print YES if the count is even, otherwise NO

Code:

Lab 2 > Contest > C++ D.cpp > ...

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  string larrysArray(int A[], int n)
6  {
7      int inv = 0;
8      for (int i = 0; i < n; i++)
9          for (int j = i + 1; j < n; j++)
10             if (A[i] > A[j])
11                 inv++;
12     return (inv % 2 == 0) ? "YES" : "NO";
13 }
14
15 int main()
16 {
17     int t;
18     cin >> t;
19     while (t--)
20     {
21         int n;
22         cin >> n;
23         int A[n];
24         for (int i = 0; i < n; i++)
25             cin >> A[i];
26         cout << larrysArray(A, n) << endl;
27     }
28     return 0;
29 }
```

Output:

```
3
3
3 1 2
YES
4
1 3 4 2
YES
5
1 2 3 5 4
NO
```

Verdict:

66640855	fardinnuman	D	Accepted	261	508	C++	10 min ago
----------	-------------	---	----------	-----	-----	-----	------------

Discussion:

- **Time Complexity:** $O(n^2)$ for counting inversions in the array
- **Space Complexity:** $O(n)$ for storing the array
- **Insights:** The key observation is that 3-element rotations preserve inversion parity, so only the parity of inversions matters
- **Difficulties:** Ensuring correct inversion counting for all test cases

E. Almost Sorted

Problem Statement:

We are given an array of distinct integers, we have to determine if it can be sorted in ascending order using at most one of two operations: a single swap of any two elements, or reversing a single contiguous subarray. If the array is already sorted, print yes. Otherwise, determine if a single swap or reverse can sort the array. If both are possible, prefer swap. If neither works, print no

Theory/Approach:

In this solution, we first check if the array is already sorted. If not, it identifies the leftmost and rightmost indices where the array is unsorted. If swapping the elements at these two positions sorts the array, then swap is the answer. Otherwise, if reversing the subarray between these indices sorts the array, then reverse is the solution. If neither operation results in a sorted array, it cannot be sorted with a single operation

Code:

```
Lab 2 > Contest > C++ Ecpp > ...
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      int n;
8      cin >> n;
9      int a[n], s[n];
10     for (int i = 0; i < n; i++)
11     {
12         cin >> a[i];
13         s[i] = a[i];
14     }
15     sort(s, s + n);
16
17     int l = 0, r = n - 1;
18     while (l < n && a[l] == s[l])
19         l++;
20     while (r >= 0 && a[r] == s[r])
21         r--;
22
23     if (l > r)
24     {
25         cout << "yes\n";
26         return 0;
27     }
28     swap(a[l], a[r]);
29     bool ok = true;
30     for (int i = 0; i < n; i++)
31         if (a[i] != s[i])
32             ok = false;
33     if (ok)
34     {
35         cout << "yes\nswap " << l + 1 << " " << r + 1 << endl;
36         return 0;
37     }
38     swap(a[l], a[r]);
39
40     reverse(a + l, a + r + 1);
41     ok = true;
42     for (int i = 0; i < n; i++)
43         if (a[i] != s[i])
44             ok = false;
45     if (ok)
46         cout << "yes\nreverse " << l + 1 << " " << r + 1 << endl;
47     else
48         cout << "no\n";
49
50     return 0;
51 }
```

Output:

```
3
3 1 2
no

6
1 5 4 3 2 6
yes
reverse 2 5
```

Verdict:

66659092	fardinnuman	E	Accepted	615	753	C++	4 min ago
----------	-------------	---	----------	-----	-----	-----	-----------

Discussion:

- **Time Complexity:** $O(n \log n)$ due to sorting the array once; all other operations are $O(n)$
- **Space Complexity:** $O(n)$ for storing the sorted array
- **Insights:** Identifying the first and last mismatch positions allows targeted swap or reverse checks

F. Birthday Cake Candles

Problem Statement:

In this problem, the heights of candles placed on a birthday cake is given, our task is to find the tallest candle height and count how many candles have that height. Only the tallest candles can be blown out, so the final count must be printed

Theory/Approach:

The solution reads the number of candles and processes each height one at a time. While reading the heights, it keeps track of the maximum height found so far. If a taller candle is found, the maximum height is updated and the count is reset. If a candle has the same height as the current maximum, the count is increased

Code:

Lab 2 > Contest > C++ F.cpp > ...

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      int n;
8      cin >> n;
9
10     int hmax = 0;
11     int count = 0;
12
13     for (int i = 0; i < n; ++i)
14     {
15         int h;
16
17         cin >> h;
18         if (h > hmax)
19         {
20             hmax = h;
21             count = 1;
22         }
23         else if (h == hmax)
24         {
25             count++;
26         }
27     }
28     cout << count << endl;
29
30     return 0;
31 }
```

Output:

```
4
3 2 1 3
2
```

Verdict:

66360332	fardinnuman	F	Accepted	178	411	C++	12 days ago
----------	-------------	---	----------	-----	-----	-----	-------------

Discussion:

- **Time Complexity:** $O(n)$, since each candle height is checked once
- **Space Complexity:** $O(1)$